

ESP-NOW Ping-Pong Link Test

How the code works - ESP32-C3 SuperMini ↔ ESP32-C6 SuperMini

ESP-NOW transport | Arduino-ESP32 core 3.x | derived from the LoRa Ping-Pong test

1 Purpose & topology

A link-quality test for two different SuperMini boards talking over **ESP-NOW** (Espressif's connectionless 2.4 GHz protocol). Each board pings the other and reports packet counts, loss, and round-trip time (RTT) on a built-in web page - no router, app, or PC tools needed. It is the LoRa Ping-Pong test with the radio layer swapped for ESP-NOW.

```
Unit 1 = ESP32-C3 SuperMini      Unit 2 = ESP32-C6 SuperMini
+-----+ ESP-NOW +-----+
| C3 (PINGPONG_ID=1) | PING ----> | C6 (PINGPONG_ID=2) |
|                   | <---- PONG |                   |
+-----+ ch 1 +-----+
AP: ESPNOW-PingPong-C3          AP: ESPNOW-PingPong-C6
http://192.168.4.1             http://192.168.4.1
```

Both boards run the **same firmware**. The PlatformIO env picks the board and sets PINGPONG_ID (1/2) and UNIT_LABEL (C3/C6); everything else is identical.

2 What changed vs the LoRa version

Aspect	LoRa Ping-Pong	ESP-NOW Ping-Pong (this)
Radio	E32-433T30S over UART	built-in 2.4 GHz ESP-NOW
Addressing	shared channel, transparent	broadcast (FF:...:FF), no pairing
Framing	scan magic byte in byte stream	whole packet in recv callback
Receive	poll UART in loop()	callback -> ring buffer -> loop()
Boards	2x XIAO C3	C3 SuperMini + C6 SuperMini
Core	Arduino-ESP32 2.0.x	3.x (required by the C6)

3 The channel rule (most important)

ESP-NOW frames are only received by radios sitting on the **same WiFi channel**. There is no scanning or negotiation. So both units pin themselves to one channel: each starts a SoftAP on ESPNOW_CHANNEL (default 1), which fixes the radio to that channel, and the broadcast peer is added on the same channel. If the two units used different channels they would never hear each other.

```
WiFi.mode(WIFI_AP);
WiFi.softAP(ssid, "12345678", ESPNOW_CHANNEL); // radio locked to ch 1
esp_now_init();
esp_now_register_recv_cb(onRecv);
peer.peer_addr = FF:FF:FF:FF:FF:FF; peer.channel = ESPNOW_CHANNEL;
peer.ifidx = WIFI_IF_AP; esp_now_add_peer(&peer);
```

Running a SoftAP also gives each unit its web portal at 192.168.4.1 on that same channel - radio sharing between ESP-NOW and the AP is fine.

4 Packet format

The same 19-byte packed struct as the LoRa test. ESP-NOW delivers whole datagrams, so no start-byte framing is needed - but the magic byte and an XOR checksum are kept to reject anything that is not one of our frames.

Field	Type	Meaning
magic	uint8	0x50 - identifies our frame
type	uint8	1 = PING, 2 = PONG
src / dst	uint8 x2	sender / target unit id
seq	uint16	sequence number (increments per ping)
t_ms	uint32	pinger's send-time (millis); echoed in PONG
src_sent	uint32	sender's total pings sent (peer monitoring)
src_rt	uint32	sender's total round trips (peer monitoring)
crc	uint8	XOR of all preceding bytes

5 Message flow, RTT & loss

```
C3 (pinger)                                C6 (responder)
t_ms = millis() --- PING(seq,t_ms) -->;
awaiting = true                            recv PING (dst==2): pingsRecv++
                                           &lt;-- PONG(seq,t_ms) --- reply, ECHOING t_ms unchanged
recv PONG (seq==awaitSeq)
RTT = millis() - t_ms // same clock stamps & measures -&gt; no sync needed
roundTrips++ ; awaiting = false
```

The responder echoes the pinger's t_ms verbatim, so when the PONG returns the pinger subtracts it from the current millis() for the full round trip. Loss: one ping is kept 'in flight' (awaiting); if the next 2 s tick arrives while still awaiting, that ping is counted lost and a fresh one is sent. Success rate = roundTrips / (roundTrips + lost).

6 Receive path - callback to ring buffer

ESP-NOW calls onRecv() in the WiFi task whenever a frame arrives. Doing real work (or sending a reply) there is risky, so it only copies the frame into an 8-slot ring buffer; loop() drains and processes it. This keeps the radio callback short and the PONG reply on the main thread.

```
onRecv(info, data, len):                    // core 3.x signature
  if len != 19: return
  copy 19 bytes into rxRing[head]; advance head (drop if full)

loop(): drainRx():
  while tail != head:
    p = rxRing[tail++]
    if magic ok and crc ok and p.src == PEER:
      PING -&gt;; send PONG(echo t_ms)          PONG -&gt;; compute RTT, roundTrips++
```

7 Web portal

setup() starts the SoftAP (ESPNOW-PingPong-<label>, password 12345678) and a WebServer on port 80 with two routes:

Route	Returns
/	Self-contained dashboard, polls /api every 1 s
/api	JSON: id, label, peer, link_up, pings_sent, round_trips, lost, pings_recv, success_pct, last/min/max/avg RTT, peer_send

Two cards - This unit and Peer (counters carried inside every packet) - plus a LINK UP/DOWN badge (up while a peer packet arrived within LINK_TIMEOUT_MS = 6 s). Either unit's page shows both units' status.

8 Boards, build, flash

Unit	Board id	Flag
------	----------	------

ESP32-C3 SuperMini	esp32-c3-devkitm-1	PINGPONG_ID=1, UNIT_LABEL=C3
ESP32-C6 SuperMini	esp32-c6-devkitm-1	PINGPONG_ID=2, UNIT_LABEL=C6

```
pio run -e c3_supermini -t upload      # ESP32-C3 SuperMini (Unit 1)
pio run -e c6_supermini -t upload      # ESP32-C6 SuperMini (Unit 2)
# prebuilt: bin/ESPNow_PingPong_C3-SuperMini_id1.bin , ..._C6-SuperMini_id2.bin
```

platform = espressif32 (Arduino-ESP32 core 3.x). The C6 is only supported on core 3.x, so this project must not pin core 2.x. Tuning knobs at the top of main.cpp: PING_INTERVAL_MS, LINK_TIMEOUT_MS, ESPNOW_CHANNEL.